Waggle: Graph-Backed Persistent Memory for AI Agents with Recursive Memory Context Assembly

Abhigyan Shekhar
Independent Research
open-source project

***Abstract*—AI agents lose all conversational context when the context window closes. Existing approaches—raw context dumps, flat retrieval-augmented generation (RAG), and note storage—fail to surface relational structure, contradictions, or superseded decisions at scale. We present Waggle, a local-first persistent memory system built on a typed knowledge graph, and Recursive Memory Context Assembly (RMCA), an algorithm that assembles compact, high-signal context packs from that graph via query decomposition, multi-lane retrieval, typed-edge expansion, conflict resolution, and token-budget compression.**

We evaluate across three complementary regimes. *(1) Session retrieval:* On LongMemEval-S (500 questions), achieves 89.0% Exact@5 and 97.4% R@5 with ; a held-out split confirms no overfitting (+1.1 pp gap). *(2) Synthetic context assembly:* On five RLM-style memory task families, RMCA achieves perfect scores on pairwise conflict reasoning and ContextReset (session-boundary state reconstruction) while flat baselines score 0.0; linear aggregation remains hard for all methods. *(3) LLM answer quality:* Groq achieves EM=1.0, F1=1.0, and hallucination rate=0.0 when given RMCA context packs on pairwise conflict tasks, consistent across 3 seeds and 2 scales; all flat baselines fail.

Ablation studies identify query decomposition as the only currently causally isolated load-bearing component; graph expansion and conflict resolution do not yet show a measurable delta due to benchmark construction limitations. Synthetic benchmark numbers are not numerically comparable to the RLM paper, which evaluates real long-context datasets with frontier models. We release code, benchmark generators, and result artifacts with the paper.

***Index Terms*—agent memory, knowledge graph, retrieval-augmented generation, context assembly, persistent memory, LLM agents, conflict resolution**

I. Introduction

AI coding assistants and conversational agents are stateless by design: each session begins with an empty context window. Practitioners work around this by pasting prior conversation summaries, maintaining hand-curated notes, or relying on the model to infer context from the current session alone. None of these approaches scale.

The core problem has three dimensions:

1) **Volume.** A long-running project accumulates thousands of decisions, constraints, and implementation facts. No context window can hold them all.
2) **Structure.** Decisions depend on constraints; constraints contradict earlier preferences; implementations derive from decisions. Flat text storage discards this relational structure.
3) **Staleness.** Decisions get reversed. Constraints get relaxed. A memory system that cannot distinguish current from superseded information will mislead the agent.

Retrieval-augmented generation (RAG) [1] addresses volume but not structure or staleness. GraphRAG-style systems [2] add community-detection graphs over document chunks but operate on pre-computed summaries and do not model temporal validity or contradiction. Episodic memory systems [3], [4] store conversation turns but do not extract typed relational structure.

We make the following contributions:

- **Waggle**, a local-first persistent memory system built on a typed knowledge graph with temporal validity, contradiction handling, and a git-inspired snapshot vocabulary.
- **RMCA (Recursive Memory Context Assembly)**, an algorithm that assembles compact context packs from the graph via query decomposition, multi-lane retrieval, typed-edge expansion, conflict resolution, and token-budget compression.
- **Empirical evaluation** on LongMemEval-S (500 questions) and five synthetic task families, with held-out splits, LLM-backed answer-level evaluation, and explicit ablation studies.
- **Open-source implementation** as an MCP-compatible server () deployable with a single .

II. Background and Related Work

A. Context Window Limitations

Transformer-based LLMs have fixed context windows. While window sizes have grown (4K $\rightarrow$ 128K $\rightarrow$ 1M tokens), the fundamental problem remains: information outside the window is inaccessible, and filling the window with raw history is expensive and degrades attention quality at long ranges [9].

B. Retrieval-Augmented Generation

RAG [1] retrieves relevant text chunks by semantic similarity and injects them into the prompt. It addresses volume but has three structural weaknesses relevant to agent memory: flat ranking cannot distinguish current from superseded decisions; chunks are independent so dependency structure is lost; and two contradictory chunks are returned with equal weight.

C. GraphRAG and Knowledge Graph Retrieval

Microsoft GraphRAG [2] builds a community-detection graph over document chunks and retrieves community summaries. This improves coverage for broad queries but precomputes summaries offline and does not model temporal validity or agent-specific relational types (decisions, constraints, contradictions).

D. Episodic and Working Memory for Agents

MemGPT [3] introduces a tiered memory architecture with explicit paging between in-context and external storage. Generative Agents [4] use a memory stream with recency, importance, and relevance scoring. These systems focus on retrieval of episodic facts; they do not model typed relational edges or contradiction resolution as first-class operations.

E. Recursive Language Models

The RLM framework [7] proposes externalising long context into an environment and interacting with it through decomposition and targeted retrieval. Waggle's RMCA algorithm is directly inspired by this decomposition-retrieval loop, applied to a persistent typed graph rather than a document corpus. The RLM paper reports that base models score $<0.1\%$ F1 on OOLONG-Pairs while structured recursive assembly recovers to 23–58% F1, a qualitative pattern consistent with our findings.

III. System Design

A. Memory Graph

Waggle stores memory in a persistent typed knowledge graph $G = (V, E)$.

Node types: , , , , , , .

Edge types: , , , , , , .

Every node carries temporal validity fields and . Nodes whose has passed are excluded from default queries. The operation accepts a node ID and sets the losing node's to the current time, making supersession explicit and reversible.

The graph is backed by SQLite with WAL mode for local-first deployments, with an optional Neo4j backend for remote or multi-client deployments.

B. Embeddings

Waggle uses [6] for local embedding with no external API dependency. The default model is (≈420 MB, 384-dimensional embeddings). A deterministic SHA-256 fallback is available for offline or resource-constrained environments.

C. Retrieval Modes

Three retrieval lanes are available:

- : semantic similarity over node embeddings, with typed-edge expansion.
- : graph retrieval fused with BM25 lexical scoring.
- : semantic search over the raw conversation transcript store.

D. MCP Tool Surface

Waggle exposes its functionality as an MCP (Model Context Protocol) server. Table I lists the six core tools used in normal agent operation.

TABLE I
Waggle MCP Tool Surface

Tool	Purpose
	Ingest a turn; extract nodes and edges
	Retrieve a subgraph for a query
	Hydrate context at session start
	Run RMCA for a full context pack
	Show what changed recently
/	Structured atomic writes

IV. Recursive Memory Context Assembly (RMCA)

RMCA is the algorithm behind . It takes a query q, the memory graph G, the transcript store T, a token budget B, expansion depth d, and maximum subquery count k, and returns a structured context pack C with $\widehat{\text{tok}}(C) \leq B \times 1.15$.

A. Algorithm

Step 1 — Decompose. The query is classified as project/coding or generic. A set of up to k targeted subqueries is generated, each with a priority weight and a set of retrieval modes. For project queries, subqueries target: recent decisions, unfinished tasks, constraints and rejected directions, implementation details, conflicts, and the original query. For generic queries, subqueries target: the original query, recent relevant facts, related decisions, contradictions, and transcript evidence.

Step 2 — Retrieve. Each subquery is issued against the graph and transcript store using its assigned retrieval modes. Results are accumulated into a working hit set H.

Step 3 — Expand. The top-5 nodes in H by score are used as seeds. For each of the top-3 seeds, 1–2 hop graph expansion is performed along typed edges (, , , ,). Expansion along and is excluded to avoid token budget inflation.

Step 4 — Resolve. All edges in H are inspected. Nodes connected by edges have their scores penalised ($\times 0.3$) and the updating node is boosted ($+0.15$). Nodes connected by edges are recorded as conflict pairs. Nodes with expired are penalised ($\times 0.2$).

Step 5 — Deduplicate. For each unique , only the highest-scored copy is retained.

Step 6 — Rank. Nodes are sorted by: superseded status (non-superseded first), score (descending), label (ascending for ties).

Step 7 — Compress. Nodes are packed into the context string C in priority order: decisions, constraints, implementation, unfinished, conflicts, superseded, evidence. Packing stops when the token budget $B \times 1.15$ is reached.

Step 8 — Format. A structured header is prepended. Provenance and conflict annotations are attached.

TABLE II
RMCA VS. TOP-k RAG

Dimension	Top-k RAG	RMCA
Query strategy	Single embedding query	Decomposed into $\leq k$ targeted subqueries
Index structure	Flat chunk index	Typed knowledge graph
Graph expansion	None	1–2 hop traversal along typed edges
Conflict handling	None	Explicit / detection
Staleness handling	None	Temporal validity () with score penalisation
Token budget	Fixed top-k	Configurable budget with priority-ordered packing
Evidence provenance	Chunk source	Verbatim transcript lane + node provenance

B. Comparison with Top-k RAG

Table II summarises the key differences between RMCA and standard top-k RAG.

C. Comparison with GraphRAG

TABLE III
RMCA VS. GRAPHRAG

Dimension	GraphRAG	RMCA
Graph construction	Community detection over document chunks	Typed relational graph from conversation turns
Query strategy	Single query against pre-computed summaries	Runtime decomposition into targeted sub-queries
Token budget	Fixed community summary size	Configurable budget with priority ordering
Verbatim evidence	Not available	Verbatim transcript retrieval lane
Temporal validity	Not modelled	First-class / on every node

V. EVALUATION

A. LongMemEval-S

Dataset. LongMemEval-S [5] is a 500-question benchmark for multi-session conversational memory retrieval. Each question requires identifying the correct support session(s) from a haystack of prior sessions. The cleaned split () contains 500 questions with gold support sets of cardinality 1–6.

Metrics. We report R@5 (recall: does the gold set appear anywhere in the top 5?), Exact@5 (precision: does the top 5 exactly match the gold set?), Exact@10, and Exact@20.

Setup. Sessions are prepared by converting each haystack session to a text block, normalising timestamps to UTC ISO format, and embedding unique session texts once with . A prepared-session cache keyed by dataset SHA-256, mode, limit, and embedding model is written on the first run and reused on subsequent runs. All reported numbers use the warm cache.

Modes. Two retrieval modes are evaluated:

- : user turns only; weighted score $= 0.72 \times$ semantic $+$ $0.18 \times$ lexical $+ 0.10 \times$ temporal; top-5 returned directly.
- : user and assistant turns; same raw ranking followed by heuristic reranking of top-20 using RRF fusion, lexical overlap, and temporal recency bias.

Results. Table IV shows the main results. achieves 89.0% Exact@5 with no tunable reranking heuristics, making it the most reproducible baseline. scores lower on Exact@5 but recovers strongly at higher cutoffs (Exact@20 = 98.0%), indicating it finds all correct sessions but does not always pack them into the top 5 slots for high-cardinality queries.

TABLE IV
LONGMEMEVAL-S RESULTS (500 QUESTIONS,)

Mode	R@5	Exact@5	Exact@10	Exact@20
	97.4%	**89.0%**	89.0%	89.0%
	97.0%	87.2%	94.8%	**98.0%**

Cardinality breakdown. Table V shows performance by gold-set cardinality. R@5 = 100% at cardinality ≥ 3 confirms the embedding reliably finds the correct base session. The Exact@5 drop at high cardinality is a packing problem: fitting 4–6 chunks into 5 slots while excluding noise is structurally harder than retrieval itself.

TABLE V
PERFORMANCE BY GOLD-SET CARDINALITY

Cardinality	n	R@5	Exact@5
1	176	95.5%	95.5%
2	250	98.0%	93.2%
3	41	100.0%	73.2%
4	19	100.0%	47.4%
5	11	100.0%	45.5%
6	3	100.0%	0.0%

Overfitting check. To verify the scores are not artefacts of the specific 500-question distribution, we ran a held-out split (50 dev / 450 test, seed 42). Table VI shows the results. generalises cleanly. The 5.3 pp gap for is expected: the heuristic reranking weights (RRF fusion, lexical/coverage scoring) are partially tuned to this distribution. Exact@5 should be treated as an exploratory number rather than a headline result.

TABLE VI
HELD-OUT SPLIT OVERFITTING CHECK (SEED 42)

Mode	Dev Exact@5 (n=50)	Test Exact@5 (n=450)	Gap
	88.0%	89.1%	+1.1 pp — stable
	92.0%	86.7%	−5.3 pp — dev-set sensitivity

B. RLM-Style Synthetic Benchmark

To evaluate RMCA's behaviour across task types that vary in structural complexity, we constructed a synthetic benchmark

suite following the task taxonomy of the RLM paper [7]. Each family maps a canonical long-context task to a Waggle memory graph scenario.

Task families. Table VII describes the five task families.

TABLE VII
SYNTHETIC BENCHMARK TASK FAMILIES

Family	RLM equiv.	What it tests
S-NIAH-style	RULER S-NIAH [9]	Retrieve one fact from N distractors — O(1)
BrowseComp-Plus-style	BrowseComp-Plus	Multi-hop QA across 3 linked nodes
OOLONG-style	OOLONG [8]	Aggregate N entries — O(n)
OOLONG-Pairs-style	OOLONG-Pairs	Pairwise conflict reasoning — O(n^2)
CodeQA-style	LongBench-v2 CodeQA [10]	Codebase/repo understanding

Methods compared.

- : dump all graph nodes into the context window (no retrieval).
- : single semantic query, top-k results returned.
- : full RMCA pipeline.

Scales. Each family is evaluated at $N = 128$, 512, and 2048 nodes.

Results. Table VIII shows the main results.

TABLE VIII
RLM-STYLE SYNTHETIC BENCHMARK RESULTS

Family	Scale	raw	query	build
S-NIAH	128	1.000	1.000	**1.000**
S-NIAH	512	1.000	1.000	**1.000**
S-NIAH	2048	1.000	1.000	**1.000**
BrowseComp+	128	1.000	1.000	**1.000**
BrowseComp+	512	1.000	1.000	**1.000**
BrowseComp+	2048	1.000	1.000	**1.000**
OOLONG-Pairs	128	0.000	0.000	**1.000**
OOLONG-Pairs	512	0.000	0.000	**1.000**
OOLONG-Pairs	2048	0.000	0.000	**1.000**
CodeQA	128	0.000	1.000	**1.000**
CodeQA	512	0.000	1.000	**1.000**
CodeQA	2048	0.000	1.000	**1.000**
OOLONG	128	0.885	0.242	0.513
OOLONG	512	0.403	0.035	0.224
OOLONG	2048	0.000	0.010	0.069

Token efficiency. On S-NIAH tasks, uses 13–14% of the tokens consumed by while achieving the same score (7–8× reduction). On OOLONG-Pairs tasks, uses ≈36% of tokens while achieving 1.0 vs. 0.0 score.

Key findings.

1) **Pairwise conflict reasoning (OOLONG-Pairs).** Both and score 0.0 at all scales. scores 1.0 at all scales. RMCA succeeds on this task because decomposition generates targeted conflict/constraint subqueries and the formatted context pack exposes the conflict relation in an answerable structure. Current ablations isolate decomposition—not graph expansion or conflict resolution—as the load-bearing component. Typed edges and conflict resolution are part of the RMCA design, but current synthetic benchmarks do not yet isolate their causal contribution.
2) **Codebase understanding (CodeQA).** scores 0.0 at all scales (the relevant code fact is buried in a large context dump). and both score 1.0, confirming that targeted retrieval is sufficient for this task type.
3) **Simple factual retrieval (S-NIAH, BrowseComp-Plus).** All three methods score 1.0 at all scales. RMCA uses 7–8× fewer tokens than raw context while achieving the same score.
4) **Linear aggregation (OOLONG).** scores highest at small scale (0.885 at N=128) but degrades to 0.0 at N=2048 as the context window fills. scores 0.513 at N=128 and degrades to 0.069 at N=2048. This exposes a fundamental limitation: tasks requiring aggregation over all N entries cannot be fully satisfied by any fixed-budget retrieval system.

C. Latency

Local latency on SQLite with deterministic embeddings (warm cache):

TABLE IX
OPERATION LATENCY (SQLITE, WARM CACHE)

Operation	Mean latency
	1.54 ms
	1.60 ms
	0.80 ms
(N=128)	≈22 ms
(N=2048)	≈258–415 ms

latency scales with graph size due to the multi-subquery retrieval loop. At N=2048, latency is in the 250–415 ms range, which is acceptable for agent use but not for real-time interactive applications.

D. LLM-Backed Answer-Level Evaluation

To validate that retrieval-level scores translate to answer quality, we conducted an LLM-backed answer-level evaluation using Groq as the answering model. Each method's context pack was injected into a structured prompt; the model's answer was evaluated for exact match (EM), F1, and hallucination rate.

Setup. Evaluation was run across 3 seeds (42, 43, 44) and 2 scales (128, 512 nodes) on the pairwise conflict task. Results were consistent across all seeds and scales.

Results. Table X shows the pairwise task results.

Key findings.

1) RMCA is the only method achieving EM=1.0, F1=1.0, and hallucination rate=0.0 on pairwise conflict tasks, consistent across all 3 seeds and both scales.
2) retrieves insufficient context (98 tokens) and the model correctly marks the question as rather than hallucinating.

TABLE X
LLM ANSWER-LEVEL EVALUATION — PAIRWISE CONFLICT TASK
(GROQ , SEEDS 42/43/44, SCALES 128/512)

Method	EM	F1	Hall.	Tokens
	1.00	**1.00**	**0.00**	435–515
	0.00	0.00	0.00	98
	0.00	0.00–0.12	0.00–1.00	1390–1422
	0.00	0.00–0.12	0.00–1.00	1390–1422
	0.00	0.00–0.12	0.00–1.00	1389–1422

3) Flat baselines (, ,) inject 1390–1422 tokens but either fail to answer correctly or hallucinate, because the conflict annotation is not surfaced in the flat context dump.
4) The pattern is consistent across seeds 42, 43, 44 and scales 128, 512, confirming the result is not a random artefact.

Caveat. These results use one answering model and should be replicated across models. The model is an answerer, not an independent human judge. The is a reproducible lower-bound evaluator; its scores are not equivalent to human preference ratings or LLM-judge quality assessments.

E. Ablation Study

To identify which RMCA components are load-bearing, we ran a systematic ablation study across 7 variants on the benchmark (conflict represented only by typed edges; no conflict vocabulary in node content).

Setup. 4 variants $\times$ 2 scales (128, 512) $\times$ 3 seeds (42, 43, 44). All seeds produced identical scores (deterministic benchmark).

Results. Table XI shows the ablation results.

TABLE XI
RMCA ABLATION STUDY — BENCHMARK
(SCALES 128/512, SEEDS 42/43/44; ALL SEEDS IDENTICAL)

Variant	S=128	S=512	Δ	Status
	1.000	1.000	—	Baseline
	0.000	0.000	-1.000	**Load-bearing**
	1.000	1.000	0.000	Not isolated
	1.000	1.000	0.000	Not isolated

Real-embedding confirmation. The ablation was repeated with Waggle's production model (384-dim sentence-transformers) to rule out artefacts of the deterministic hash-based embedding. Table XII shows the results.

TABLE XII
ABLATION WITH REAL EMBEDDINGS ()
VS. DETERMINISTIC EMBEDDINGS

Variant	Det.	Real	Diff.
	1.000	1.000	0.000
	1.000	1.000	0.000
	1.000	1.000	0.000
	0.000	0.000	0.000

Root cause analysis. The real embedding model produces identical results to the deterministic model because the root cause is *benchmark construction*, not the embedding model. The node labels (e.g., "Cloud database", "Local deployment required") are semantically distinctive enough that retrieves the relevant choice and constraint nodes directly via cosine similarity, without needing to traverse the edge.

To causally isolate graph expansion and conflict resolution, the benchmark would need nodes where: (1) choice and constraint labels are semantically similar to distractors, so that semantic retrieval alone cannot distinguish them; and (2) the only signal distinguishing the gold conflict pair is the edge. This requires a fundamentally different benchmark design.

Summary. Decomposition is the only RMCA component causally isolated as load-bearing on the current synthetic benchmarks. Graph expansion and conflict resolution may be load-bearing in real-world settings, but the current benchmark cannot demonstrate this due to construction limitations.

F. ContextReset Benchmark

The ContextReset benchmark evaluates whether a memory system can correctly reconstruct project state after a context window reset. It tests six fields: decision recall, constraint recall, next-step accuracy, superseded context handling, active decision preference, and evidence coverage.

Setup. Scale N=128, after the decomposition fix (short continuation queries now use broad project-state subqueries).

Results. Table XIII shows the results.

TABLE XIII
CONTEXTRESET BENCHMARK RESULTS (N=128)

Method	Score	Tokens
	1.000	315
	0.875	96
	0.000	1413
	0.000	1413
	0.000	0

achieves a perfect score of 1.000 across all six evaluation fields. scores 0.875, missing one field due to insufficient context breadth. Both and score 0.0 despite injecting 1413 tokens, because the flat context dump does not surface the structured project state needed to answer the multi-field question. scores 0.0 as expected.

The key insight from this benchmark is that structured context assembly (RMCA) outperforms both flat retrieval and raw context dump on multi-aspect project-state reconstruction tasks, using only 22% of the tokens consumed by the flat baselines.

G. Comparison with RLM Paper

The RLM paper [7] reports results on the real OOLONG and OOLONG-Pairs datasets using frontier models. Table XIV shows the comparison.

Note: Rows in this table are not a leaderboard. The RLM paper evaluates real long-context datasets (OOLONG

, OOLONG-Pairs) with frontier models (GPT-5, Qwen3-Coder-480B). Waggle evaluates synthetic graph-memory analogues with a 70B open model. Numbers are not comparable. The shared qualitative pattern is that structured decomposition/assembly outperforms flat/base methods on pairwise/conflict-style tasks.

TABLE XIV
QUALITATIVE ALIGNMENT WITH RLM-STYLE FINDINGS
(NOT NUMERICALLY COMPARABLE — NOT A LEADERBOARD)

System	OOLONG	OOLONG-Pairs F1	Dataset
RLM (GPT-5) [7]	≈68%	58.0%	Real
RLM (Qwen3-Coder-480B) [7]	≈73%	23.1%	Real
Base models [7]	—	<0.1%	Real
Waggle RMCA (scoped)	55%	—	Synthetic (20 cases)
Waggle RMCA (scoped fullscan)	100%	—	Synthetic (20 cases)
Waggle RMCA (pairwise)	—	**100%**	Synthetic (9 nodes)

Important caveats.

1) The Waggle synthetic OOLONG results (55% scoped, 100% scoped-fullscan) are *not comparable* to the RLM paper's 68–73% on the real split. The datasets, models, and scales differ fundamentally.
2) The OOLONG real-30 run was interrupted by API token-per-day limits before completion; the 20-case synthetic result is preliminary.
3) The RMCA pairwise F1 of 100% is on a synthetic task with 9 nodes and explicit edges, which is substantially simpler than the real OOLONG-Pairs split.
4) The qualitative pattern is consistent: structured assembly outperforms flat retrieval on conflict-aware tasks. Base models score <0.1% F1 on OOLONG-Pairs while structured recursive assembly recovers to 23–58% F1 in the RLM paper; RMCA achieves 100% F1 on the synthetic pairwise task.

H. Supported and Unsupported Claims

Table XV summarises which claims are supported by current evidence and which require further work. This table is intended to help reviewers assess the scope of our contributions.

VI. DISCUSSION

A. When RMCA Helps

RMCA provides the largest benefit over raw context and single-query retrieval on tasks that require:

- **Conflict and contradiction detection.** On current pairwise benchmarks, query decomposition is the primary causally isolated load-bearing component. Disabling decomposition or replacing it with random subqueries drops score from 1.0 to 0.0. Disabling graph expansion or explicit conflict resolution does not reduce score, because direct retrieval already surfaces the relevant conflict nodes in this synthetic setup. Typed edges and conflict resolution are part of the RMCA design, but current synthetic benchmarks do not yet isolate their causal contribution.
- **Multi-aspect information needs.** Query decomposition (Step 1) allows RMCA to retrieve memory relevant to decisions, constraints, implementation details, and conflicts in a single call, where a single-query retrieval would only surface the most semantically similar nodes.
- **Token-constrained contexts.** At large graph sizes, raw context dumps exceed any practical context window. RMCA's token-budget compression (Step 7) ensures the context pack fits regardless of graph size.
- **Project-state reconstruction.** The ContextReset benchmark demonstrates that RMCA's structured assembly correctly reconstructs multi-field project state (decisions, constraints, next steps, superseded context) where flat retrieval fails.

B. When RMCA Does Not Help

RMCA does not improve over simpler methods on:

- **Simple factual retrieval.** When the answer is a single node with high semantic similarity to the query, is sufficient and faster.
- **Linear aggregation.** Tasks requiring enumeration of all N entries are fundamentally O(n) information needs. No fixed-budget retrieval system can guarantee full coverage. RMCA degrades more gracefully than raw context at large scale but does not solve this problem.

C. Limitations

1) **Synthetic benchmark data.** The RLM-style benchmark uses deterministic synthetic Waggle memory tasks. Numerical results must not be compared to the RLM paper or other long-context benchmarks until the exact public datasets are run with a matching model setup. The OOLONG real-30 run was interrupted by API token-per-day limits before completion; the 20-case synthetic result (55% scoped, 100% scoped-fullscan) is not comparable to the RLM paper's 68–73% on the real split.
2) **Token estimation is approximate.** $\widehat{\text{tok}}(s) = \lfloor |s|/4 \rfloor$ may differ from actual tokenizer counts by $\pm 20\%$.
3) **Decomposition is heuristic, not learned.** Subquery generation uses keyword pattern matching and does not use an LLM. It may produce suboptimal decompositions for queries outside the project/coding and generic-memory categories.
4) **Graph expansion is bounded.** Expansion is limited to 2 hops from the top-3 seed nodes. Deep reasoning chains requiring more than 2 hops are not guaranteed to be surfaced.
5) **Graph expansion and conflict resolution are not yet causally isolated.** Ablation studies on both the standard and the harder benchmark (conflict represented only by typed edges, no conflict vocabulary in node

TABLE XV
SUPPORTED AND UNSUPPORTED CLAIMS

Claim	Status	Evidence	Caveat
RMCA improves pairwise answerability	**Supported**	Groq eval: EM/F1=1.0, hall=0.0 across 3 seeds × 2 scales	Synthetic pairwise task; one answer model
Decomposition is load-bearing	**Supported**	and drop 1.0→0.0	Current synthetic tasks only
RMCA helps ContextReset	**Supported**	=1.0 vs =0.875 and raw/bm25/no_memory=0.0	Synthetic project-state benchmark
RMCA reduces tokens vs raw context	**Supported (selected tasks)**	S-NIAH: 13–14% of raw tokens; pairwise: ≈31–38%	Not universally lower than
Graph expansion is load-bearing	**Not supported yet**	Ablation shows no delta at scales 128/512	Benchmark construction may not isolate edge traversal
Conflict resolution is load-bearing	**Not supported yet**	Ablation shows no delta at scales 128/512	Direct retrieval already surfaces conflict nodes
RMCA solves linear aggregation	**Not supported**	OOLONG-style degrades with scale	O(n) coverage under fixed budget
Results numerically comparable to RLM paper	**Not supported**	Different datasets, models, scales	Qualitative comparison only
Results generalise to real-world traces	**Not supported yet**	No real trace benchmark	Future work

content) show zero delta when graph expansion or conflict resolution is disabled, using both the deterministic hash-based embedding model and Waggle's production model. The root cause is benchmark construction: node labels are semantically distinctive enough that direct retrieval surfaces conflict nodes without edge traversal. Isolating these components requires a benchmark with semantically indistinguishable node labels and a scorer that requires explicit conflict annotation in the context pack.

6) **LLM answer-level evaluation uses a single model.** The Groq answer-level results use one answering model and should be replicated across models. A second-model run with was deferred because was not available in the evaluation environment. The model is an answerer, not an independent human judge. The is a reproducible lower-bound evaluator; its scores are not equivalent to human preference ratings or LLM-judge quality assessments.
7) **LongMemEval scope.** The adapter evaluates session retrieval only. End-to-end graph extraction quality, contradiction handling across sessions, and context-bundle usefulness for model handoff are not yet measured.
8) **reranking sensitivity.** The 5.3 pp dev/test gap confirms the hybrid reranking weights are partially tuned to this distribution and should not be treated as a portable headline number.
9) **OOLONG-Pairs comparison is qualitative, not quantitative.** The RLM paper reports RLM(GPT-5) at 58.0% F1 and RLM(Qwen3-Coder-480B) at 23.1% F1 on the real OOLONG-Pairs split, while base models score <0.1%. RMCA achieves 100% F1 on the synthetic pairwise task (9 nodes, explicit conflict edges). The qualitative pattern is consistent—structured assembly outperforms flat retrieval on conflict-aware tasks—but the synthetic task is substantially simpler than the real OOLONG-Pairs split and the underlying models differ.

VII. CONCLUSION

We presented Waggle, a local-first persistent memory system for AI agents, and RMCA, an algorithm for assembling compact, high-signal context packs from a typed knowledge graph.

On LongMemEval-S, achieves 89.0% Exact@5 with no overfitting confirmed by a held-out split (+1.1 pp gap), demonstrating strong session retrieval on a real multi-session benchmark.

On synthetic pairwise conflict reasoning and ContextReset tasks, RMCA achieves perfect scores while flat baselines score 0.0. LLM-backed answer-level evaluation confirms EM=1.0, F1=1.0, and hallucination rate=0.0 for RMCA on pairwise tasks across 3 seeds and 2 scales. Ablation studies identify **query decomposition as the only currently causally isolated load-bearing component**: disabling it drops pairwise score from 1.0 to 0.0. Graph expansion and conflict resolution remain plausible contributors to RMCA's design, but current synthetic benchmarks do not yet isolate their causal contribution due to benchmark construction limitations.

On linear aggregation tasks, RMCA degrades gracefully but exposes a fundamental O(n) coverage limit shared by all fixed-budget retrieval systems.

The key insight is that agent memory is not a retrieval problem alone—it is a retrieval-plus-reasoning problem. Retrieving the right facts is necessary but not sufficient; the memory system must also surface which facts are current, which are superseded, and which are in tension with each other. RMCA provides the architectural machinery for this,

but causal isolation of individual components remains an open empirical question.

Future work includes: (1) replacing heuristic query decomposition with a learned decomposer; (2) evaluating on the full RLM public benchmark suite with a matching model setup; (3) measuring end-to-end graph extraction quality on LongMemEval; (4) hardening the hybrid reranking weights against distribution shift; (5) constructing a variant with semantically indistinguishable node labels to causally isolate graph expansion and conflict resolution; and (6) evaluating on real agent traces, which is the most important next step for establishing external validity.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A Graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[3] C. Packer, S. Wooders, K. Lin, V. Fang, S. I. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[4] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[5] X. Wu, H. Wang, W. Xiong, C. Hu, and W. Y. Wang, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," *arXiv preprint arXiv:2410.10813*, 2024.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[7] A. L. Zhang, T. Kraska, and O. Khattab, "Recursive language models," *arXiv preprint arXiv:2512.24601*, 2026.

[8] A. Bertsch, A. Pratapa, T. Mitamura, G. Neubig, and M. R. Gormley, "OOLONG: Evaluating long context reasoning and aggregation capabilities," *arXiv preprint arXiv:2511.02817*, 2025.

[9] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, and B. Ginsburg, "RULER: What's the real context size of your long-context language models?" *arXiv preprint arXiv:2404.06654*, 2024.

[10] Y. Bai, X. Lv, J. Zhang, Y. He, J. Qi, L. Hou, J. Tang, Y. Dong, and J. Li, "LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks," *arXiv preprint arXiv:2412.15204*, 2025.

APPENDIX

1) **DECOMPOSE:** Classify query intent; generate $\leq k$ subqueries $\{sq_i\}$ with priority weights and retrieval modes.
2) **RETRIEVE:** $H \leftarrow \bigcup_i \text{retrieve}(sq_i, G, T)$ — issue each subquery against graph and transcript store.
3) **EXPAND:** seeds $\leftarrow$ top-5 nodes in H; for each $s \in$ seeds[:3]: $H \leftarrow H \cup \text{graph\_hop}(s, G, \min(d, 2))$.
4) **RESOLVE:** For edges: penalise target ($\times 0.3$), boost source ($+0.15$). For edges: record conflict pair. For expired nodes: penalise ($\times 0.2$).
5) **DEDUPLICATE:** $H \leftarrow \{\arg\max_{\text{score}} h : h.\text{node\_id}\}$ — keep highest-scored copy per node.
6) **RANK:** Sort H by (is_superseded $\uparrow$, score $\downarrow$, label $\uparrow$).
7) **COMPRESS:** Pack sections [decisions, constraints, impl., unfinished, conflicts, superseded, evidence] until $\widehat{\text{tok}}(C) > B \times 1.15$.
8) **FORMAT:** Prepend structured header; attach provenance and conflict annotations.
   Return C, provenance(H), conflict_entries.

A. LongMemEval-S

B. RLM-Style Synthetic Benchmark

C. Ablation Study